Pairs-Trading Statistical Arbitrage: Project Report

Author: Shayan Ponnanna Malchira
Date: October 16, 2026
Language and tools: Python 3.10+, pandas, numpy, statsmodels, scikit-learn, matplotlib, yfinance (alpaca-py for the optional live module)

1. Objective

This project builds an end-to-end statistical arbitrage pipeline. It scans a universe of stocks from one sector, finds cointegrated pairs, backtests a mean-reversion strategy out of sample with realistic trading costs, and produces a daily BUY / SELL / HOLD signal for the best pairs. The emphasis is on methodological correctness and honest evaluation rather than on producing an attractive backtest.

2. Data

- **Universe:** 22 liquid large-cap US banks, defined in a configuration file (an energy universe is also provided). The universe is not hardcoded in the logic.
- **Source:** Split- and dividend-adjusted daily closing prices from yfinance, cached locally as CSV.
- **Cleaning:** Tickers with more than 5% missing data are dropped, short gaps (up to 3 days) are forward-filled, and any remaining incomplete rows are removed.
- **Windows:** Training from 2019-01-01 to 2022-12-31. Testing from 2023-01-01 to the latest available date. The code asserts that the two windows do not overlap.

3. Methodology

3.1 Pair selection (training window only)

1. For every pair of stocks, run the Engle-Granger cointegration test on log prices.
2. Keep pairs with p-value below 0.05.
3. Estimate the hedge ratio and intercept by OLS. The spread is log(Y) - beta * log(X) - alpha. Pairs with beta <= 0 are discarded so that every pair is a genuine long/short trade.
4. Estimate the half-life of mean reversion from an AR(1) fit of the spread. Keep pairs with a half-life between 2 and 60 days.
5. Rank the remaining pairs by the sum of their p-value rank and half-life rank, and keep the top 5.

Hedge ratios are estimated once on the training window and are never refit during testing.

3.2 Trading rules

The signal uses a trailing 60-day z-score of the spread.

Event	Rule
Enter long spread (BUY)	z < -2
Enter short spread (SELL)	z > +2
Exit	|z| < 0.5
Stop-out	|z| > 4
Time stop	40 trading days in the trade

After a stop-out or time stop, the strategy does not re-enter until the spread has returned inside the exit band. Each signal translates into two stock positions:

- **BUY (long spread):** buy $1 of stock Y and short $beta of stock X.
- **SELL (short spread):** short $1 of stock Y and buy $beta of stock X.
- **HOLD:** stay flat and close any open position.

3.3 Backtest design

- **Execution:** A signal generated at the close of day *t* is filled at the close of day *t+1*.
- **Costs:** 5 basis points per side on the traded notional of every leg. This is configurable and is included in every reported figure.
- **Capital:** Capital equals gross notional (no leverage). Borrow costs are not modelled.
- **Portfolio:** Equal capital is allocated to each selected pair. All selected pairs are reported, whether profitable or not.
- **Metrics:** Total return, annualized Sharpe ratio (risk-free rate of 0, 252 days, computed over all test days including flat days), maximum drawdown, number of trades, win rate, and turnover.
- **Charts:** Equity curves per pair and for the portfolio, plus a spread and z-score chart per pair with entries and exits marked.

3.4 Optional machine-learning entry filter

A logistic regression is trained on the training window only to predict whether a spread entry will mean-revert within 20 days before hitting the stop level.

- **Features:** z-score, absolute z-score, 5-day change in z-score, 20-day spread volatility, and pair half-life.
- **Use:** An entry is taken only if the predicted probability exceeds the training-set base rate.
- **Evaluation:** The backtest is run with and without the filter on the test window and the difference is reported as-is, including if the filter does not help.

3.5 Optional live module

A paper-trading module reads the latest signals and prints the orders it would place through Alpaca. It defaults to dry-run mode, and API keys are read from environment variables only, never from code.

4. Safeguards Against Bias

- **Lookahead bias:** Pair selection, hedge ratios, and the ML model use only training data. The z-score uses a trailing window only, and a unit test confirms that past values do not change when future data is added.
- **Execution realism:** Signals are filled one day after they are generated, not at the price that triggered them.
- **Cherry-picking:** All selected pairs are reported, and parameters were fixed before looking at test-window performance.
- **Multiple testing:** The pipeline prints how many pairs were tested and how many would be expected to pass the $p < 0.05$ threshold by chance alone.

5. Software Structure and Testing

File	Purpose
config.py	Universe, dates, thresholds, costs
data.py	Download, cache, and clean prices
pairs.py	Cointegration test, hedge ratio, half-life, ranking
strategy.py	Spread, z-score, signal state machine

File	Purpose
backtest.py	Cost-aware backtest, metrics, charts
ml_filter.py	Optional logistic-regression entry filter
live.py	Optional Alpaca dry-run order generation
main.py	Runs the full pipeline with one command
tests.py	Unit tests

The unit tests cover causality of the z-score, the entry/exit state machine, cost accounting, per-trade PnL, and the half-life estimator.

6. Limitations

- A single train/test split gives a single draw of out-of-sample performance, which is not proof of a durable edge.
- Testing many pairs produces false positives by chance.
- Hedge ratios are frozen, so relationship drift during the test window hurts performance.
- The universe consists of today's large banks, which introduces survivorship bias.
- The cost model is a flat 5 bps and ignores borrow fees, market impact, and wider spreads in stressed markets.
- The strategy uses daily closes only, with no intraday risk modelling.
- The ML filter is trained on in-sample spreads, which tend to look more mean-reverting than out-of-sample ones.

7. Why Any Edge May Not Persist Live

- Cointegration between same-sector stocks is regime-dependent, and events such as rate shocks or bank-specific stress can break spreads permanently.
- Statistical arbitrage is a crowded strategy, so well-known relationships tend to be arbitraged away.
- Live frictions such as borrow availability, partial fills, and slippage are typically larger than backtest assumptions.
- Pairs that rank best in training are partly the luckiest ones, which biases expectations upward.
- Stop-outs cluster in stress periods, when correlations spike and spreads diverge together.